The Smart-Attend Mobile App

1. Task 1: Waterfall Failure Analysis

Why Waterfall Model Would Cause a Crisis

The **Waterfall Model** follows a **linear and sequential development process** where each phase must be completed before the next begins. In the *Smart-Attend* project, the sudden requirement of **Biometric Face ID verification** midway through development would create major problems.

1. Rigidity in Requirements

In the Waterfall model, requirements are defined at the beginning and rarely changed afterward. Once development starts, it becomes difficult to modify the system design.

In this project, the app was originally designed only for **Geo-fencing attendance**. When the Dean introduces the **Face ID verification requirement**, the team would have to revisit earlier phases such as requirement analysis and system design. Waterfall does not support such backward movement easily, causing delays and confusion.

2. High Cost of Change

Waterfall requires detailed documentation and design before coding begins. If a major feature like **biometric authentication** is introduced midway, the following must be redesigned:

- System architecture
- Database structure
- Mobile app UI
- Authentication logic
- Security mechanisms

These changes increase development cost, delay the project timeline, and may require rewriting large parts of the code.

3. Delayed Testing and Integration Issues

In Waterfall, testing usually happens **after the full development phase**. If biometric verification is integrated late in the process, the team may discover issues such as:

- Compatibility problems with mobile devices
- Security vulnerabilities
- Incorrect facial recognition
- Performance issues with real-time verification

Because testing occurs late, fixing these issues becomes expensive and time-consuming.

Conclusion

Due to its rigid structure, high cost of changes, and late testing cycle, the Waterfall model would struggle to accommodate the sudden **Face ID requirement**, potentially causing project delays and budget overruns.

---

2. Task 2: Agile Pivot Using Scrum Framework

To handle changing requirements effectively, the team can adopt the **Agile Scrum framework**, which supports iterative development and continuous feedback.

Instead of building the entire system at once, the project is divided into **small development cycles called sprints**.

Each sprint lasts **2 weeks** and delivers a working version of the application.

Sprint 1 (Minimum Viable Product – MVP)

Goal: Deliver the **basic working attendance system**.

Features included:

1. Basic mobile app interface (UI)
2. Student login using university ID
3. Geo-fencing attendance detection
4. Automatic attendance marking
5. Teacher dashboard showing present students

Outcome:
Students can enter the classroom and their attendance will automatically be recorded based on location.

Sprint 2 (Feature Update)

Goal: Improve security and enhance system functionality.

Features included:

1. **Biometric Face ID verification** to prevent proxy attendance
2. Integration of facial recognition with attendance system
3. Improved teacher/admin reporting dashboard
4. Bug fixes from Sprint 1 feedback
5. Performance optimization

Outcome:
Attendance will only be confirmed when **both geo-fencing and face verification succeed**, ensuring accurate and secure attendance tracking.